

Universita degli Studi dell'Insubria

Laboratorio Interdisciplinare A

CineMax

Manuale Tecnico

Architettura, struttura del codice, gestione file e scelte implementative

Autori	Oliver Munger, Davide Noe Velasquez Carpio, Davide Gatti
Versione documento	1.0
Data	17/08/2026
Corso	Laboratorio Interdisciplinare A

Indice

1. Scopo e contesto tecnico
2. Architettura generale
3. Struttura del repository
4. Package e classi
5. Modello dati
6. Gestione file CSV
7. Flussi principali
8. Algoritmi e controlli
9. Gestione errori
10. Compilazione e test
11. Limiti tecnici
12. Bibliografia

Cronologia del documento

Versione	Data	Autore	Descrizione
1.0	17/08/2026	Team CineMax	Documento completo fatto sulla base del progetto Java e dei requisiti di consegna.

Scopo del documento

Questo manuale descrive CineMax dal punto di vista tecnico. È destinato a docenti, revisori e sviluppatori che devono capire struttura del progetto, scelte implementative e gestione dei dati.

Materiale di riferimento

- Specifiche del progetto CineMax: applicazione Java per cinema monosala da 200 posti, gestione palinsesto, prenotazioni e ruoli utente.
- Linee guida di documentazione: manuale utente ad alto livello, manuale tecnico con architettura, strutture dati, algoritmi e formato file.
- Repository consegnato: cartelle src, data, doc, file README.txt e autori.txt.
- Nota: la parte JavaDoc non viene trattata in questo manuale, come richiesto.

1. Contesto tecnico

CineMax è un'applicazione Java console-based per la gestione di un cinema monosala. Il progetto usa una struttura a package, classi di dominio e una classe di utilità per la persistenza su file. Non sono presenti database relazionali, interfacce grafiche o architetture client/server.

Vincolo/Scelta	Implementazione attuale
Linguaggio	Java, con main in cinemax.CineMax.
Interfaccia	CLI/TUI tramite Scanner e System.out.
Persistenza	File CSV nella cartella data.
Ruoli	Cliente, Proiezionista, Bigliettaio, tutti sottoclassi di Utente.
Database	Non presente. Tutte le operazioni sono su file.
Concorrenza	Non gestita, coerente con un uso locale monoutente.

2. Architettura generale

L'architettura è organizzata in livelli logici semplici: avvio applicazione, controllo menu, utenti/ruoli, classi di dominio e gestione file. L'obiettivo è mantenere separata la logica di interazione con il terminale dalla rappresentazione dei dati e dalle operazioni di I/O.

Livello	Classi principali	Responsabilità
Avvio	CineMax	Crea Scanner, mostra menu iniziale, indirizza registrazione/login/guest.
Controllo menu	GestoreMenu	Gestisce registrazione cliente, login e sessioni di ruolo.
Utenti	Utente, Cliente, Proiezionista, Bigliettaio	Rappresentano i ruoli e le azioni disponibili per ciascun utente.
Dominio	Film, Genere, Proiezione, Prenotazione	Rappresentano film, categorie, spettacoli e biglietti/prenotazioni.
Persistenza	FileManager	Legge e scrive CSV, aggiorna posti, password hash, ricerca dati su file.

Schema testuale del flusso generale:

```
CineMax.main
-> menu iniziale
    -> registrazione cliente -> FileManager.salvaUtente
    -> login -> FileManager.caricaUtentePerUsername -> sessione ruolo
    -> guest -> ricerca proiezioni -> FileManager.caricaPalinsesto

Sessione ruolo
    Cliente -> prenotazioni e ricerche
    Proiezionista -> gestione palinsesto
    Bigliettaio -> ricerca prenotazioni
```

3. Struttura del repository

La cartella fornita contiene la seguente struttura principale.

```
LabA-Va-Munger-Velasquez-Gatti-/
  README.txt
  autori.txt
  src/
    cinemax/
      CineMax.java
      Users/
      controls/
      gestione/
      utils/
  data/
    film.csv
    palinsesto.csv
    prenotazioni.csv
    proiezioni.csv
    utenti.csv
  doc/
    JavaDoc/
```

Elemento	Stato rilevato	Uso
src	Presente	Codice sorgente Java.
data	Presente	File CSV usati dall'applicazione.
doc/JavaDoc	Presente	Non trattata in questo manuale
README.txt	Presente	Istruzioni di esecuzione tramite IDE.
autori.txt	Presente	Dati autori e matricole.
bin/.jar	Presente	

4. Package e classi

Package	Classe	Responsabilita principale
cinemax	CineMax	Classe principale. Contiene main e menu iniziale.
cinemax.controls	GestoreMenu	Gestisce flussi di registrazione, guest, login e avvio sessioni.
cinemax.Users	Utente	Classe astratta base con dati comuni e metodi astratti di menu.
cinemax.Users	Cliente	Operazioni cliente: ricerca, prenotazione, visualizzazione, modifica, cancellazione.
cinemax.Users	Proiezionista	Operazioni su palinsesto: aggiunta, modifica, eliminazione proiezioni.
cinemax.Users	Bigliettaio	Consultazione e ricerca prenotazioni.
cinemax.gestione	Film	Dati descrittivi del film.
cinemax.gestione	Genere	Enum dei generi selezionabili con codice e descrizione.
cinemax.gestione	Proiezione	Dati di una proiezione: ID, data, ora, prezzo, film, posti disponibili.
cinemax.gestione	Prenotazione	Dati del biglietto/prenotazione e codici univoci.
cinemax.utils	FileManager	Gestione CSV, hash password, aggiornamento posti, caricamento dati.

5. Modello dati

Il modello dati è composto da entità semplici, collegate principalmente tramite oggetti Java e ID salvati nei file CSV.

Entita	Attributi rilevanti	Note
Utente	nome, cognome, username, passwordHash, dataNascita, luogoDomicilio	Classe astratta, specializzata nei ruoli.
Film	titolo, genere, regista, anno, durata, età minima	Usato da Proiezione.
Proiezione	idProiezione, data, ora, prezzo, film, postiDisponibili	Capienza iniziale pari a 200 posti.
Prenotazione	idPrenotazione, cliente, proiezione, numeroPosto, codiceBiglietto	ID e codice biglietto generati con UUID abbreviato.
Genere	codice, descrizione	Enum con mapping numero-descrizione per menu testuali.

6. Gestione file CSV

Tutti i file operativi sono referenziati da FileManager con percorsi relativi alla radice del progetto. Per questo motivo il programma deve essere avviato dalla cartella principale, non da una sottocartella.

```
./data/utenti.csv
./data/palinese.csv
./data/prenotazioni.csv
./data/film.csv
```

6.1 Formati dei file

File	Formato riga	Note tecniche
utenti.csv	nome,cognome,username,passwordHash,dataNascita,luogoDomicilio,ruolo	Ruolo in maiuscolo: CLIENTE, PROIEZIONI STA, BIGLIETTAIO
film.csv	titolo,genere,regista,anno,durata,etaMin	I titoli possono essere racchiusi tra virgolette. Controllo duplicati case-insensitive.
palinese.csv	idProiezione,data,ora,titolo,genere,regista,anno,durata,etaMin,prezzo,postiDisponibili	File operativo delle proiezioni usato dal codice.
prenotazioni.csv	idPrenotazione,nomeCliente,cognomeCliente,usernameCliente,passwordHash,idProiezione,numeroPosto,codiceBiglietto	Può essere vuoto inizialmente. Viene popolato dalle prenotazioni clienti.

6.2 Parsing CSV

Per i file che possono contenere titoli o nomi con virgole, il codice usa in alcuni punti una split con espressione regolare che separa solo le virgole esterne alle virgolette. In altri punti viene usata una split semplice sulla virgola: questa scelta funziona con il dataset attuale, ma richiede attenzione se vengono inseriti campi contenenti virgole non protette.

6.3 Aggiornamento file

Le operazioni di aggiunta vengono svolte in append. Le operazioni di modifica o cancellazione ricostruiscono il contenuto: il codice legge le righe, modifica o esclude quella interessata e riscrive il file, talvolta usando un file temporaneo.

7. Flussi principali

7.1 Avvio e login

```
Utente sceglie Log In
-> GestoreMenu.gestisciLogin
-> FileManager.caricaUtentePerUsername
-> FileManager.verificaPassword
-> creazione sottoclasse corretta in base al ruolo
-> avviaSessioneUtente
```

7.2 Ricerca proiezioni

```
FileManager.caricaPalinsesto
-> costruzione List<Proiezione>
-> Utente.cercaProiezione
-> filtri: titolo, genere, data inizio, data fine, prezzo min, prezzo max
-> visualizzazione risultati
```

7.3 Creazione prenotazione

```
Cliente.creaPrenotazione
-> verifica posti disponibili
-> new Prenotazione(cliente, proiezione)
-> Proiezione.prenotaPosto
-> FileManager.salvaPrenotazione
-> FileManager.aggiornaPostiProiezioneSuFile
```

7.4 Gestione palinsesto

```
Proiezionista.aggiungiProiezione
-> raccolta dati data/ora/prezzo/film
-> recupero o creazione Film
-> FileManager.aggiungiProiezione
-> controllo sovrapposizione oraria
-> append su palinsesto.csv
```

7.5 Ricerca prenotazione bigliettotaio

```
Bigliettaio.cercaPrenotazione
-> carica palinsesto
-> carica prenotazioni e associa idProiezione a Proiezione
-> applica criterio: codice, nome/cognome, titolo film, intervallo date
-> visualizza dettaglio
```

8. Algoritmi e controlli principali

Area	Controllo/algoritmo	Motivazione
Username	Lettura sequenziale di utenti.csv e confronto case-insensitive.	Garantire username univoco in registrazione.
Ricerca proiezioni	Filtraggio lineare su List<Proiezione> con condizioni progressive.	Semplice e adeguato a dataset piccoli.
Sovrapposizione proiezioni	Confronto intervalli: nuovoInizio < fineEsistente e nuovoFine > inizioEsistente.	Evitare due proiezioni nella stessa sala nello stesso intervallo.
Posti disponibili	Decremento del campo postiDisponibili nel palinsesto.	Tenere aggiornata la capienza residua.
Modifica/elimina proiezione	Consentita solo se postiDisponibili è pari a 200.	Impedire modifiche a spettacoli già prenotati.
ID prenotazione	UUID abbreviato a 8 caratteri.	Generare un codice identificativo semplice per il biglietto.
Codice biglietto	Prefisso QR- più UUID abbreviato.	Generare un codice di controllo distinto dall'ID prenotazione.

Estratto logico del controllo sovrapposizione:

```
nuovoFine = nuovoInizio + durataNuovoFilm
fineEsistente = inizioEsistente + durataEsistente
se stessaData e nuovoInizio < fineEsistente e nuovoFine > inizioEsistente:
    rifiuta inserimento
```

9. Gestione errori e validazione input

Il programma effettua controlli sui principali input utente e intercetta eccezioni di parsing o I/O. I messaggi vengono mostrati direttamente a terminale.

Input/operazione	Validazione	Errore gestito
Menu numerici	Parsing con Integer.parseInt.	NumberFormatException.
Date	Formato gg/mm/aaaa con parser rigido o SimpleDateFormat non lenient.	Date inesistenti o formato errato.
Ora	LocalTime.parse in formato hh:mm.	Formato ora non valido.
Prezzo	Double.parseDouble con virgola convertita in punto.	Valore non numerico o negativo.
Anno film	Quattro cifre e range 1895-2100.	Anno non verosimile.
File CSV	Controllo esistenza e lunghezza campi.	File mancante, riga malformata, IOException.

10. Compilazione ed esecuzione

Il README suggerisce l'esecuzione da IDE aprendo src/cinemax/CineMax.java. In alternativa, dalla radice del progetto si può compilare manualmente con javac.

```
mkdir build
javac -encoding UTF-8 -d build src/cinemax/*.java src/cinemax/Users/*.java
src/cinemax/controls/*.java src/cinemax/gestione/*.java src/cinemax/utils/*.java
java -cp build cinemax.CineMax
```


11. Limiti tecnici e punti da rifinire

- I file CSV sono usati come archivio diretto: non è prevista concorrenza o transazione atomica completa.
- Alcune split CSV sono semplici: evitare virgole non protette nei campi testuali, oppure uniformare il parser.
- Il salvataggio password usa un salt/chiave costante: per un progetto didattico è accettabile, ma in produzione sarebbe preferibile un salt per utente.
- La generazione del posto assegna il numero in base ai posti rimanenti; verificare il comportamento desiderato dopo cancellazioni e modifiche.
- La console può mostrare in modo differente alcuni caratteri speciali in base alla codifica del terminale.
- Nessuna gestione per la traduzione dei titoli dei film dalla lingua originale all'italiano e viceversa.

12. Bibliografia e sitografia

- Specifiche di progetto CineMax, Laboratorio Interdisciplinare A.
- Linee guida Documentazione di Progetto, Laboratorio Interdisciplinare A.
- Documentazione ufficiale Java: `javac`, `java`, `java.time`, `java.nio.file`, `javax.crypto`.